Mentimeter

Laboratorul 2

Constructori și referințe

Mentimeter

Constructori

- funcții apelate automat la crearea unui obiect
- putem introduce restricții de integritate

Mentimeter

```java
public class Student {
    private String name;
    private Integer year;

    public Student(String newName, Integer newYear) {
        name = newName;
        year = newYear;
    }
}
```

Exemplu

Mentimeter

Constructorii:

- au numele clasei
- nu returnează explicit nici un tip (nici măcar void)
- poate avea oricâți parametri

Mentimeter

Exemplu crearea de instanțe

- Student student = new Student("of", 2);
- warning: instanță vs clasă not the same thing

Mentimeter

Constructorul implicit

- creat automat de Java dacă nu am definit noi deja unul
- inițializează câmpurile la 0/null/false în funcție de tipul lor
- nu mai este vizibil dacă am definit alți constructori

Mentimeter

Cuvântul cheie *this*

→ instanța curentă a clasei

Mentimeter

```java
public class Student {
    private String name;
    private Integer year;

    public Student(String name, Integer year) {
        this.name = name;
        this.year = year;
    }
}
```

Diferențierea între câmpurile și parametrii cu același nume

Mentimeter

```java
public class Student {
    String name;
    int year;

    public Student(String name, int year) {
        this.name = name;
        this.year = year;
    }

    public void goToSecretary() {
        Secretary secretary = new Secretary();
        secretary.handleStudent(this);
    }
}

class Secretary {
    public void handleStudent(Student student) {
        System.out.println("Oh no, here we go again " + student.name);
    }
}
```

Pasarea ca argument a referinței către instanța curentă

Mentimeter

```java
public class Student {
    String name;
    int year;

    public Student(String name, int year) {
        this.name = name;
        this.year = year;
    }

    public Student() {
        this( name: "incognito",  year: 0);
    }
}
```

Reutilizarea celorlalți constructori

Mentimeter

```java
public class Student {
    String name;
    int year;

    public Student(String name, int year) {
        this.name = name;
        this.year = year;
    }

    public Student(Student student) {
        this.name = student.name;
        this.year = student.year;
        /*
         * saaau si mai scurt:
         *     this(student.name, student.year);
         */
    }
}
```

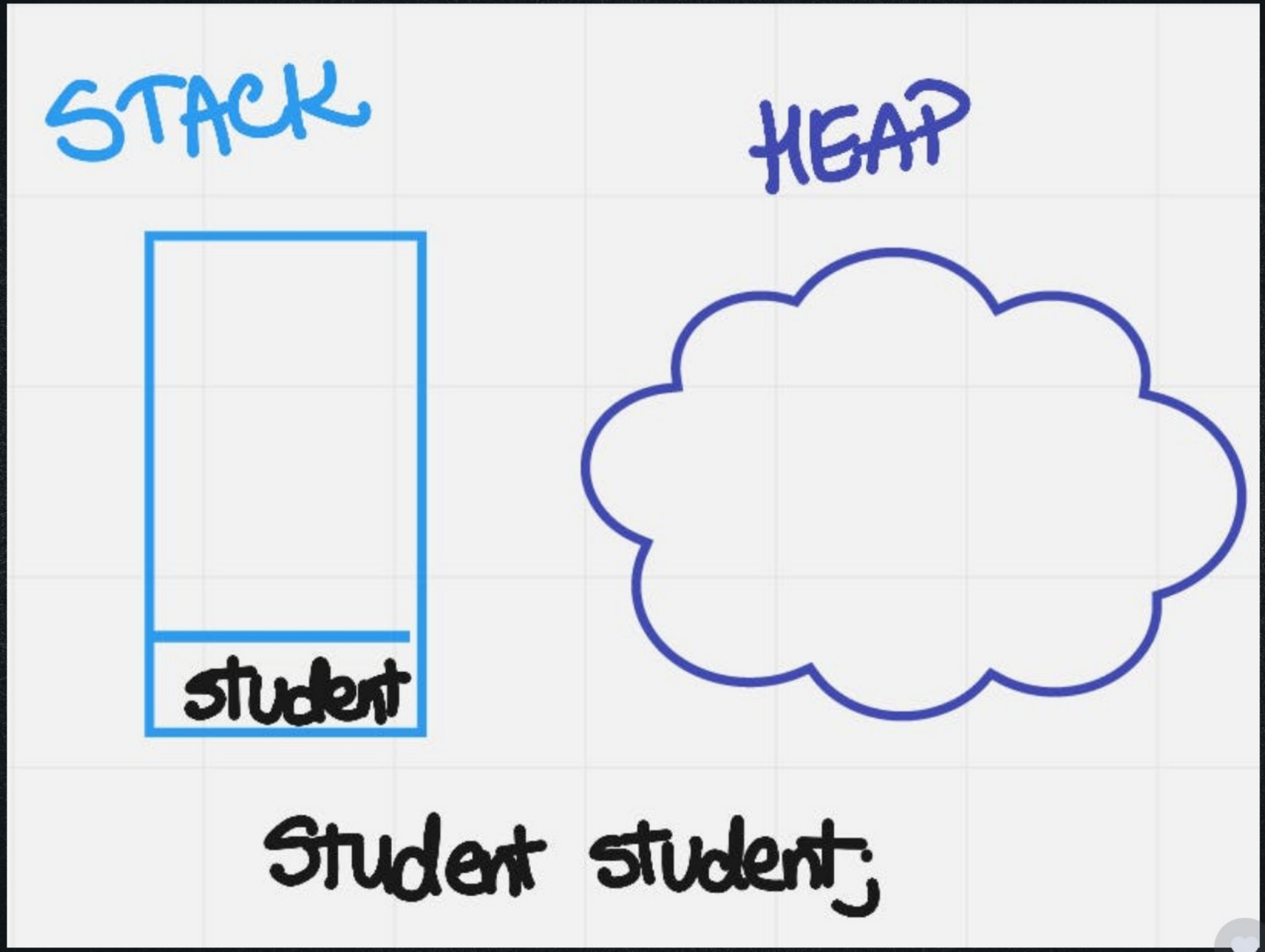
Copy constructor

Mentimeter

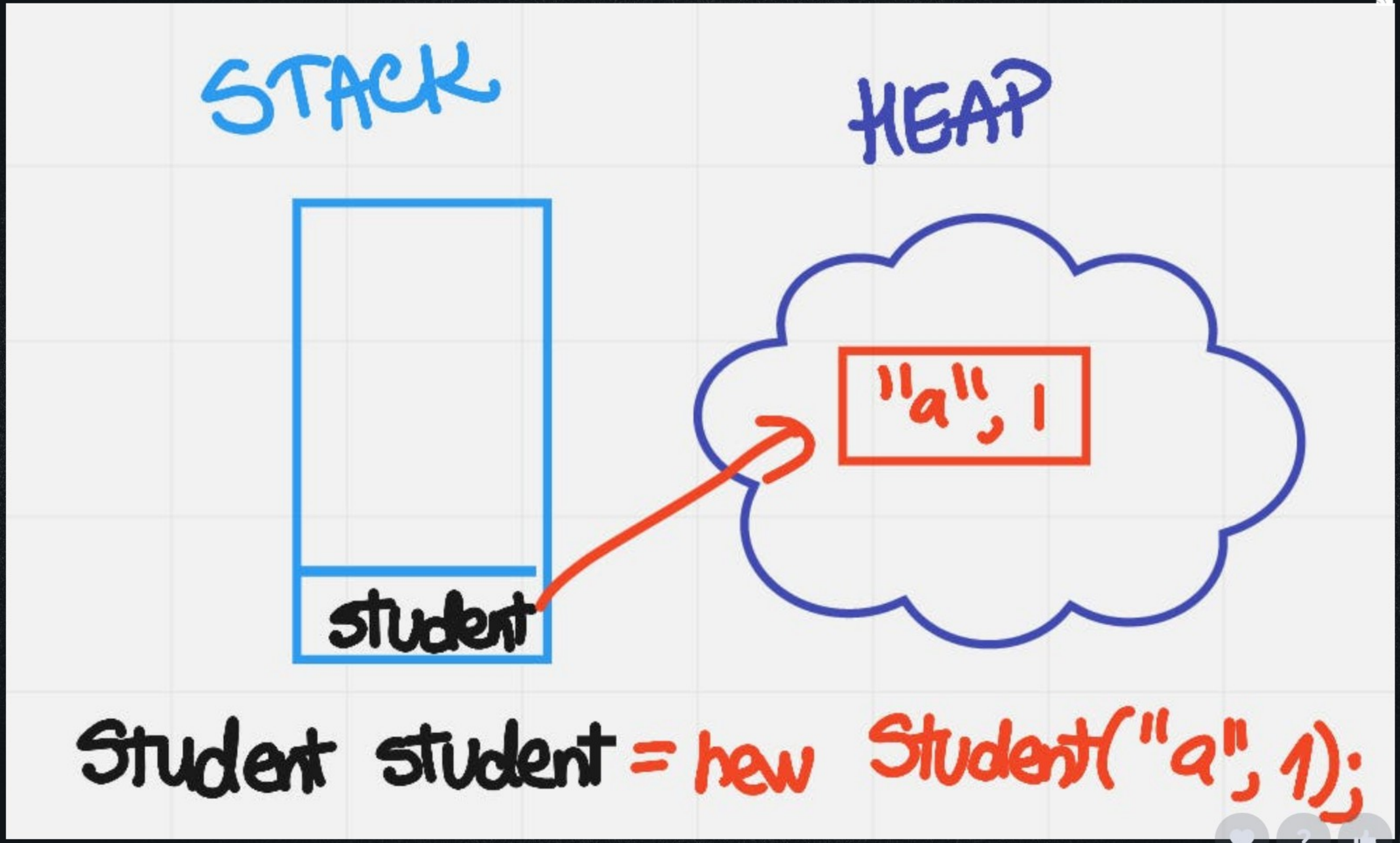
Referințe

- nu avem pointeri, avem referințe
- obiecte alocate pe heap
- referințe pe stivă
- eliberarea automată a resurselor asigurată de Garbage Collection

Mentimeter



Mentimeter



Mentimeter

```java
class Secretary {
    public void handleStudent(Student student, int availableTime) {
        System.out.println("Oh no, here we go again " + student.name);

        if (availableTime > 0) {
            System.out.println("Okay, fine");
            student.isChecked = true;
            availableTime--;
        } else {
            System.out.println("Come back later, I don't have time now");
        }
    }
}

class Main {
    public static void main(String[] args) {
        // All students are unchecked initially
        Student student1 = new Student( name: "lucky",  year: 1);
        Student student2 = new Student( name: "unlucky",  year: 4);
        Secretary secretary = new Secretary();
        int availableTime = 1;

        secretary.handleStudent(student1, availableTime);
        System.out.println(student1.isChecked);
    }
}
```

Mentimeter

true

false

Mentimeter

```java
class Secretary {
    public void handleStudent(Student student, int availableTime) {
        System.out.println("Oh no, here we go again " + student.name);

        if (availableTime > 0) {
            System.out.println("Okay, fine");
            student.isChecked = true;
            availableTime--;
        } else {
            System.out.println("Come back later, I don't have time now");
        }
    }
}

class Main {
    public static void main(String[] args) {
        // All students are unchecked initially
        Student student1 = new Student( name: "lucky",  year: 1);
        Student student2 = new Student( name: "unlucky",  year: 4);
        Secretary secretary = new Secretary();
        int availableTime = 1;

        secretary.handleStudent(student1, availableTime);
        System.out.println(student2.isChecked);
    }
}
```

Mentimeter

true

false

Mentimeter

```java
class Secretary {
    public void handleStudent(Student student, int availableTime) {
        System.out.println("Oh no, here we go again " + student.name);

        if (availableTime > 0) {
            System.out.println("Okay, fine");
            student.isChecked = true;
            availableTime--;
        } else {
            System.out.println("Come back later, I don't have time now");
        }
    }
}

class Main {
    public static void main(String[] args) {
        // All students are unchecked initially
        Student student1 = new Student( name: "lucky",  year: 1);
        Student student2 = new Student( name: "unlucky",  year: 4);
        Secretary secretary = new Secretary();
        int availableTime = 1;

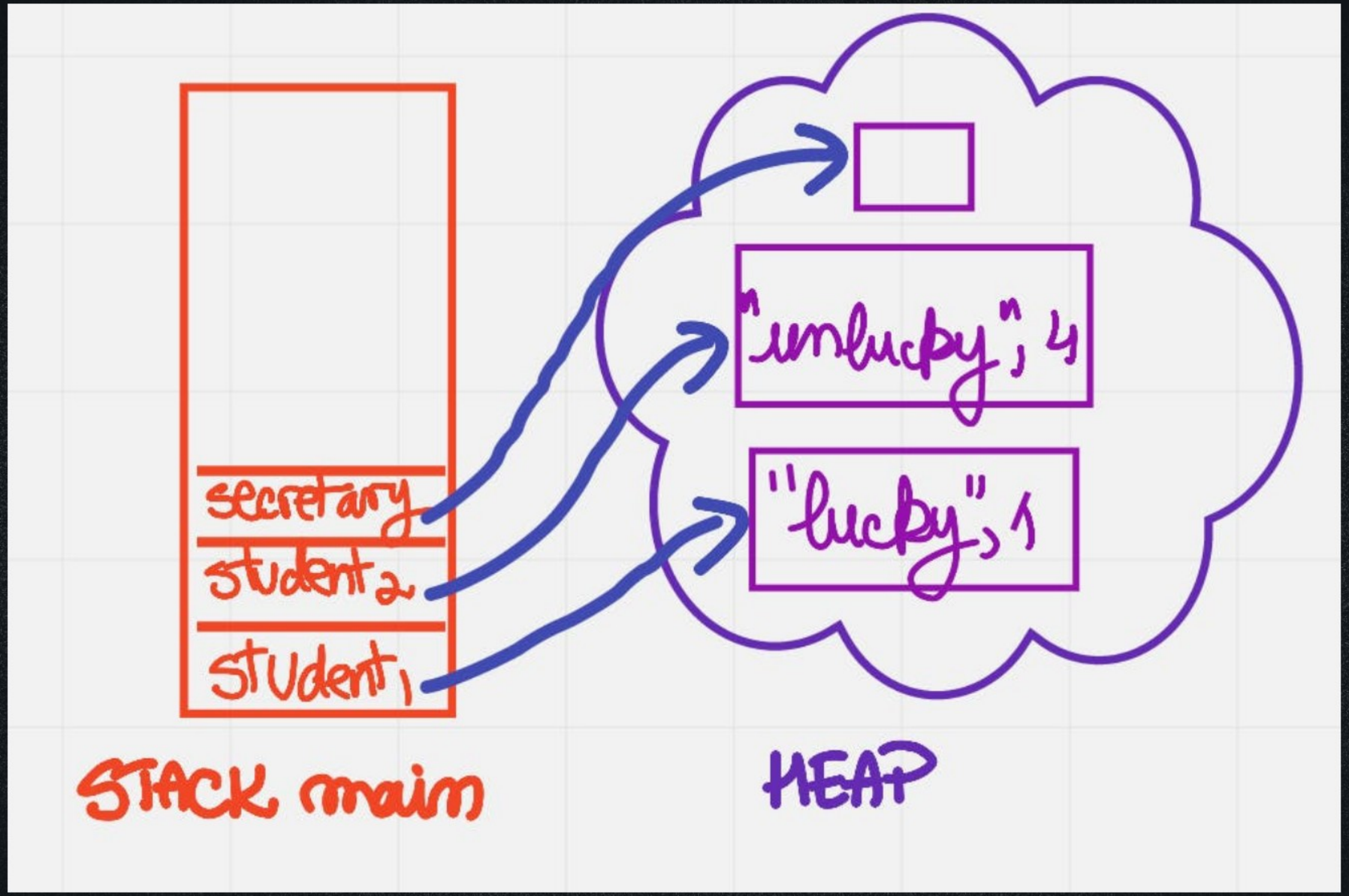
        secretary.handleStudent(student1, availableTime);
        secretary.handleStudent(student2, availableTime);
        System.out.println(student2.isChecked);
    }
}
```

Mentimeter

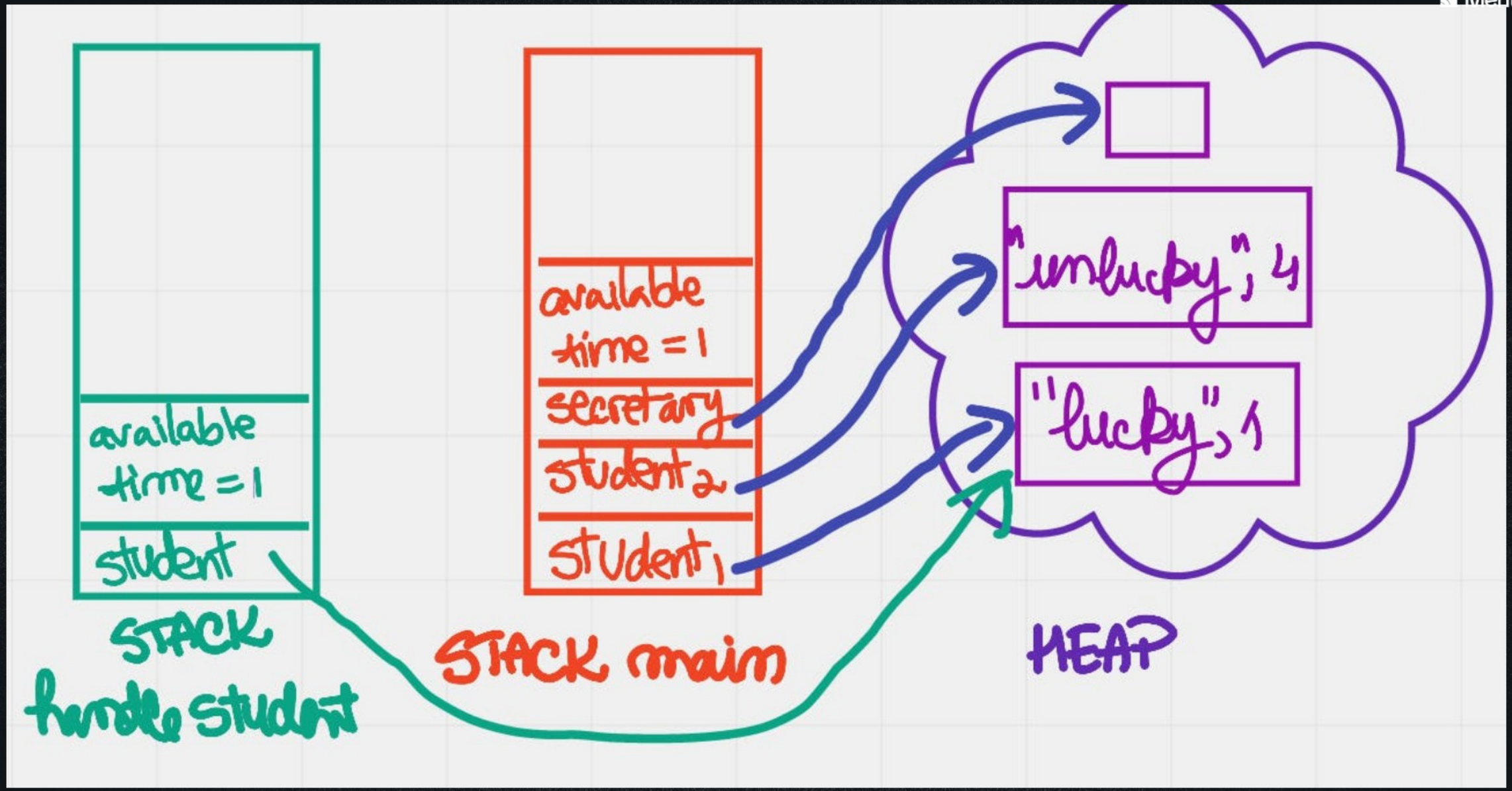
true

false

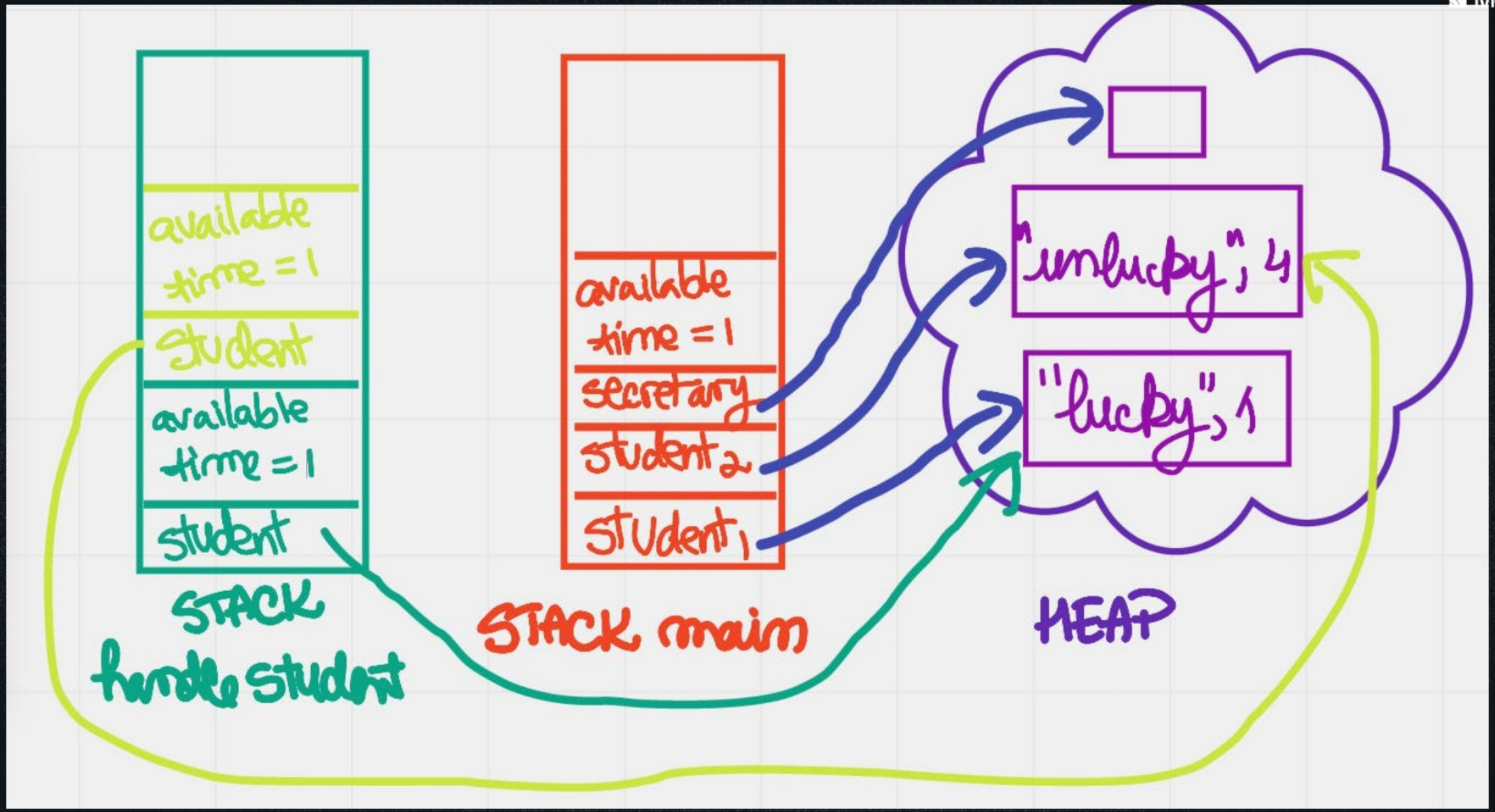
Mentimeter



Explicatie 1



Explicatie 2



Explicatie 3

Mentimeter

Transferul parametrilor

- tipurile primitive sunt copiate pe stiva => modificarile asupra lor nu sunt vizibile
- la instante se copiaza referintele => modificarile sunt vizibile

Mentimeter

Metoda toString

- implementare predefinita in Java
- putem avea implementari proprii fiecarei clase

Mentimeter

```java
public class Student {
    String name;
    int year;

    public Student(String name, int year) {
        this.name = name;
        this.year = year;
    }

    @Override
    public String toString() {
        return "Student with name " + name + " and year " + year + "\n";
    }
}
```

Exemplu toString()

Mentimeter

Demo